Paper: Spark Meets MPI: Towards High-Performance Communication Framework for Spark using MPI

Rajpreet Singh

1. Spark's fault tolerance and elasticity are never addressed or tested.
2. The comparison isn't fair (Spark version mismatch, SparkUCX missing, no error bars)
3. A minimally invasive, portable design
4. Unclear where the speedup comes from

Jiaqi Han

1. MPI4Spark-Basic failed because polling MPI from Netty's selector loop burned the CPU. The Optimized design avoids polling by sending only the bulk bodies over MPI, triggered by headers parsed in ChannelHandlers. Does that solve the event-driven vs. application-driven mismatch, or just reduce how often it happens?
2. MPI4Spark doesn't help with every workload. How could it apply MPI automatically, only when needed?
3. Why did MPI4Spark modify Netty, instead of Spark's shuffle layer like earlier work?

Qixuan Zhong

1. Does MPI4Spark preserve the existing confidentiality and integrity guarantees expected from Spark, or could it unintentionally expose sensitive data?
2. How fair is the comparison with competing systems?

and the Intel HiBench [21]. We could not collect numbers with SparkUCX because hangs were encountered with both benchmark suites. Table IV details the different benchmarks

The following library versions were used for the experiments: Apache Spark v3.3.0-SNAPSHOT, Netty v4.1.67, Apache Hadoop v3.2.3 [23], OSU HiBD Benchmarks (OHB) v0.9.3, Intel HiBench v8.0-SNAPSHOT, and v7.0 — for Spark RDMA as it uses Spark v2.1.0 — and MVAPICH2-X v2.3.6 [9].

3. How reliable are the reported improvements without measurements of variability?

Hairuo Liu

1. The Optimized design sends headers over sockets but bodies over MPI — how is partial delivery detected if the MPI body fails after the socket header arrives?
2. Without an ablation study or same-hardware TCP baseline, how do we separate MPI software speedup from RDMA hardware speedup?
3. Small messages show almost no improvement — is this JNI overhead, and was it measured?

Xingyu Wang

1. A TCP/IP data transfer is decoupled to data block and header. Then there is a natural dependency of receiving header before data block => this serialization cannot be guaranteed with two protocols using the same interconnect.
2. The performance gain is workload dependent, for frequent small communication workload, a performance degrade is expected?
3. The optimized version relies on the operation to decide whether MPI is used for data blocks. This means future API development of spark needs to adopt such a protocol, which hurts the ability to generalize.